Employee Management System: Architecture

Candidate: Kaushik Sanjay M

Overview

The Employee Management System is a three-tier web application. A React single-page app (the frontend) talks to a Spring Boot REST API (the backend), which stores data in a MySQL database. The frontend and backend communicate only through JSON over HTTP.

- Frontend: React with Vite, running on localhost:5173 in development.
- Backend: Java and Spring Boot, running on localhost:8080.
- Database: MySQL, running on localhost:3306.

Request flow: the browser calls the API through an axios client, the Controller receives the request, the Service applies the business rules, the Repository reads or writes the database through JPA and Hibernate, and the result returns to the browser as JSON.

Backend: Spring Boot

The backend uses a layered architecture. Each layer has one job, and calls only go downward from Controller to Service to Repository.

Layer	Class	Responsibility
Controller	EmployeeController	Defines the REST endpoints under /api/employees, validates request bodies with @Valid, and returns a ResponseEntity with the correct HTTP status (201 for create, 200 for the others). Contains no business logic.
Service	EmployeeService	Holds the business rules: trims and lowercases emails, prevents duplicate emails on create and update, throws ResourceNotFoundException for unknown IDs, and limits page and size values for pagination.
Repository	EmployeeRepository	A Spring Data JPA interface. Provides duplicate-email checks and a paginated search query

Layer	Class	Responsibility
		that filters by name and status.
Entity	Employee	Maps to the employee table and carries validation annotations for required fields and email format.

Supporting classes:

- DuplicateEmailException and ResourceNotFoundException are custom exceptions thrown by the service so that failures return clean HTTP errors with a message instead of stack traces.
- ApiResponse is a small wrapper that gives create, update and delete a consistent response of the form message plus data.
- Configuration such as the database URL, credentials, server port and allowed CORS origins is read from environment variables, so no secrets are stored in the code.

REST API

Method	Endpoint	Purpose	Success status
GET	/api/employees	Retrieve a page of employees. Supports page, size, search and status parameters.	200
GET	/api/employees/{id}	Retrieve one employee by ID.	200
POST	/api/employees	Create a new employee.	201
PUT	/api/employees/{id}	Update an existing employee.	200
DELETE	/api/employees/{id}	Delete an employee.	200 with confirmation message

Error responses: 400 for validation failures, 404 when an employee does not exist, and 409 when the email already exists. Each returns a readable message.

Pagination, name search and status filtering are performed in the database query, so the application stays fast as the number of employees grows.

Database: MySQL

A single employee table stores the records. It has a primary key on the employee ID and a unique constraint on email, which protects against duplicates even if the service check is bypassed. The schema

script and the required SQL queries (active employees, employees per department, name search, duplicate emails) are in the database folder of the repository.

Frontend: React

The frontend keeps UI code separate from API code, in three layers.

- api folder: httpClient.js creates one shared axios instance and converts every failure (server down, timeout, error response) into a single ApiError with a readable message. employeeApi.js contains the endpoint calls and maps the backend JSON to the shape the UI uses.
- hooks folder: useEmployees owns the current page of employees, the loading and error state, and the add, edit and delete actions. It reloads after each change so the table always matches the server. useDebounce delays search requests until the user stops typing.
- components folder: small reusable components. EmployeeTable, EmployeeForm (used for both add and edit), Pagination, SearchBar, StatusFilter, ConfirmDialog, StatusBadge, Modal, Toast, and the loading, empty and error state messages. Components never call the API directly.

App.jsx holds the page state (current page, page size, search term and status filter) and passes it to the hook. Changing the search, filter or page size returns the user to page 1.

Frontend error handling

- Forms validate required fields and email format before sending a request.
- Backend errors such as a duplicate email appear on the matching form field.
- Other failures show a message with a Try again button, so an API failure never crashes the application.

Configuration and environment variables

Both applications are configured through .env files, so database credentials and addresses are never written into the source code. Each project has a .env file for real values, which is excluded from Git, and a .env.example file that is committed as a template.

Backend (backend/.env)

The Spring Boot application reads these variables on startup. Only DB_PASSWORD must be set; every other variable has a working default in application.properties.

Variable	Default	Required	Purpose
SERVER_PORT	8080	No	Port the API runs on
DB_URL	jdbc:mysql://localhost:3306/employee_management?createDatabaseIfNotExist=true	No	MySQL connection address
DB_USERNAME	root	No	MySQL username

Variable	Default	Required	Purpose
DB_PASSWORD	none	Yes	MySQL password; the app will not start without it
JPA_DDL_AUTO	update	No	Lets Hibernate create or update tables without deleting data
JPA_SHOW_SQL	true	No	Prints SQL statements in the console for debugging
CORS_ALLOWED_ORIGINS	http://localhost:5173	No	Frontend addresses allowed to call the API (comma-separated)

Frontend (frontend/.env)

The React application reads these variables through Vite. Only variables that start with VITE_ are available to the app, and their values are visible in the browser, so no secrets are stored here.

Variable	Default	Purpose
VITE_API_BASE_URL	/api	Address prefix the frontend uses for every API call
VITE_PROXY_TARGET	http://localhost:8080	Where the Vite development server forwards /api requests (the Spring Boot backend)

During development the browser calls /api on the React server, which forwards the request to the backend. This avoids CORS problems. If the frontend is deployed on a different address from the API, VITE_API_BASE_URL is set to the full API URL and that address is added to CORS_ALLOWED_ORIGINS on the backend.

Design decisions

- Layered backend: keeps controllers thin, puts business rules in one place, and lets data access change without affecting the API.
- Server-side pagination and filtering: avoids loading every employee into the browser.
- Environment variables for configuration: keeps credentials out of source control and makes the app easy to run in different environments.
- Development proxy: the Vite dev server forwards /api requests to the backend, so the browser does not hit CORS problems during development.
- Single reusable form: one EmployeeForm handles both add and edit, which avoids duplicated validation logic.